

ui.leaflet 全面详解

`ui.leaflet` 是 NiceGUI 基于 Leaflet JavaScript 库封装的交互式地图组件，提供地图渲染、标记点、图形绘制、图层管理等核心功能，支持对接各类地图服务（如 OpenStreetMap、WMTS、WMS），适用于快速开发网页端地理信息可视化应用。以下从核心特性、配置参数、核心功能、进阶用法等维度展开全面解析。

一、核心定位与基础特性

1. 核心功能

- 地图基础控制：支持中心点设置、缩放调节、平移拖拽（可配置禁用）。
- 多源地图服务：默认集成 OpenStreetMap，支持 WMTS（Web 地图瓦片服务）、WMS（Web 地图服务）接入，可自定义地图样式。
- 图层管理：支持标记点（marker）、图像叠加层（image_overlay）、视频叠加层（video_overlay）、矢量图层（圆、多边形等）的添加、删除与编辑。
- 交互能力：地图点击事件、绘制工具条（支持多边形、矩形、标记点等绘制）、图层编辑（移动、修改、删除）。
- 扩展性：支持加载 Leaflet 插件，通过额外资源（CSS/JS）扩展功能；可直接调用 Leaflet 原生地图 / 图层方法。

2. 基础用法

通过 `ui.leaflet()` 创建地图实例，配置中心点、缩放级别等基础属性，结合按钮、标签等组件实现交互控制。示例代码结构如下：

```
from nicegui import ui

# 创建地图（中心点为伦敦坐标，缩放级别13）
m = ui.leaflet(center=(51.505, -0.09), zoom=13)

# 绑定地图状态显示（中心点、缩放级别）
ui.label().bind_text_from(m, 'center', lambda c: f'中心点: {c[0]:.3f}, {c[1]:.3f}')
ui.label().bind_text_from(m, 'zoom', lambda z: f'缩放级别: {z}')

# 按钮控制地图中心切换
ui.button('切换到柏林', on_click=lambda: m.set_center((52.520, 13.405)))

ui.run()
```

二、关键配置参数

`ui.leaflet` 的初始化参数用于定义地图基础行为与样式，以下是完整参数说明：

参数名	类型	说明	默认值	版本特性
center	元组 (float, float)	地图初始中心点 (纬度, 经度)	(0.0, 0.0)	-
zoom	整数	初始缩放级别 (Leaflet 缩放范围通常为 0-18, 0 为全球视图)	13	-
draw_control	布尔值 / 字典	是否显示绘制工具条: - 布尔值控制显示与否; - 字典可配置绘制类型 (如多边形、标记点) 及编辑权限	False	-
options	字典	传递给 Leaflet 地图的额外配置 (如禁用缩放、拖拽)	{}	-
hide_drawn_items	布尔值	是否隐藏绘制的图形 (适用于自定义渲染绘制结果)	False	2.0.0 版本新增
additional_resources	列表	额外加载的资源 (CSS/JS 文件 URL), 用于集成 Leaflet 插件	None	2.11.0 版本新增

常用 options 配置示例

通过 options 参数可禁用地图的平移、缩放等交互功能：

```
# 禁用所有交互控制
map_options = {
    'zoomControl': False, # 隐藏缩放控件
    'scrollWheelZoom': False, # 禁用滚轮缩放
    'doubleClickZoom': False, # 禁用双击缩放
    'boxZoom': False, # 禁用框选缩放
    'keyboard': False, # 禁用键盘控制
    'dragging': False, # 禁用拖拽平移
}
ui.leaflet(center=(51.505, -0.09), options=map_options)
```

三、地图样式与图层管理

1. 地图样式切换

默认地图样式为 OpenStreetMap，可通过 clear_layers() 移除默认图层，再通过 tile_layer (WMTS) 或 wms_layer (WMS) 切换自定义样式。支持从 [Leaflet Providers](#) 获取更多地图样式。

示例 1: WMTS 地图 (OpenTopoMap 地形地图)

```
m = ui.leaflet(center=(51.505, -0.09), zoom=3)
m.clear_layers() # 移除默认 OpenStreetMap 图层
m.tile_layer(
    url_template=r'https://{s}.tile.opentopomap.org/{z}/{x}/{y}.png',
    options={
        'maxZoom': 17, # 最大缩放级别
        'attribution': '地图数据版权说明' # 必须保留的版权声明
    }
)
```

示例 2: WMS 地图 (Mundialis 地理数据服务)

```
m = ui.leaflet(center=(51.505, -0.09), zoom=3)
m.clear_layers()
m.wms_layer(
    url_template='http://ows.mundialis.de/services/service?',
    options={'layers': 'TOPO-WMS,OSM-Overlay-WMS'} # 指定加载的图层
)
```

2. 核心图层类型

(1) 标记点 (Marker)

- 基础用法: 通过 `m.marker(latlng=(纬度, 经度))` 添加标记点。
- 移动标记: 通过 `marker.move(新纬度, 新经度)` 动态更新位置。
- 自定义图标: 通过 `run_method` 调用 Leaflet 原生方法修改图标。

示例: 添加可移动的标记点

```
m = ui.leaflet(center=(51.505, -0.09))
marker = m.marker(latlng=m.center) # 初始位置为地图中心

# 按钮控制标记点移动
ui.button('移动标记点', on_click=lambda: marker.move(51.51, -0.09))

# 按钮切换标记点图标
custom_icon = 'L.icon({iconUrl: "https://leafletjs.com/examples/custom-icons/leaf-green.png"})'
ui.button('更换图标', on_click=lambda: marker.run_method(':setIcon', custom_icon))
```

(2) 图像叠 layers (Image Overlay)

将图片叠加到地图指定区域, 适用于显示自定义地图、规划图等。需指定图片 URL 和边界范围 (左下角、右上角坐标)。(2.17.0 版本新增)

示例:

```
m = ui.leaflet(center=(52.5165, 13.4047), zoom=13)
m.image_overlay(
    url='https://images.squarespace-
cdn.com/content/v1/5b3e152e620b8559f2edcf7d/1613743643304-
Y7SLCT43BQ2N8C2QA3JN/1660+berlin+%28Custom%29.jpg',
    bounds=[[52.5088, 13.3877], [52.5242, 13.4218]], # 图片边界（纬度1，经度1）到（纬度2，经度2）
    options={'opacity': 0.8} # 透明度（0-1）
)
```

(3) 视频叠 layers (Video Overlay)

将视频叠加到地图指定区域，支持自动播放、透明度设置。（2.17.0 版本新增）

示例：

```
m = ui.leaflet(center=(23.0, -115.0), zoom=3)
m.video_overlay(
    url='https://www.mapbox.com/bites/00188/patricia_nasa.webm',
    bounds=[[32, -130], [13, -100]], # 视频边界
    options={'opacity': 0.8, 'autoplay': True, 'playsInline': True}
)
```

(4) 矢量 layers (Generic Layer)

通过 `generic_layer` 支持圆、多边形、折线等矢量图形，需指定 layers 类型和参数。

示例：添加红色圆形（半径 300 米）

```
m = ui.leaflet(center=(51.505, -0.09)).classes('h-64')
# 类型为 circle, 参数: 中心点、配置（颜色、半径）
m.generic_layer(name='circle', args=[m.center, {'color': 'red', 'radius': 300}])
```

3. 图层操作方法

方法	说明
<code>clear_layers()</code>	移除地图上所有 layers
<code>remove_layer(layer)</code>	移除指定 layers 实例
<code>run_map_method(name, *args)</code>	调用 Leaflet 原生地图方法（如 <code>fitworld()</code> 适配全球视图）
<code>run_layer_method(layer_id, name, *args)</code>	调用指定 layers 的原生方法（如修改透明度、图标）

示例：适配全球视图

```
m = ui.leaflet(center=(51.505, -0.09)).classes('h-64')
ui.button('显示全球', on_click=lambda: m.run_map_method('fitworld'))
```

四、交互功能

1. 地图点击事件

通过 `m.on('map-click', 回调函数)` 监听地图点击事件，获取点击位置的经纬度。

示例：点击地图添加标记点

```
from nicegui import events

m = ui.leaflet(center=(51.505, -0.09)).classes('h-96')

def handle_click(e: events.GenericEventArguments):
    # 从事件参数中提取经纬度
    lat = e.args['latlng']['lat']
    lng = e.args['latlng']['lng']
    # 添加标记点并显示提示
    m.marker(latlng=(lat, lng))
    ui.notify(f'添加标记点: ({lat:.3f}, {lng:.3f})')

m.on('map-click', handle_click)
```

2. 绘制功能 (Draw Control)

通过 `draw_control` 参数启用绘制工具条，支持绘制多边形、矩形、标记点、圆等图形，并可配置编辑和删除权限。

示例：基础绘制功能

```
from nicegui import events

def handle_draw_create(e: events.GenericEventArguments):
    # 监听绘制完成事件
    layer_type = e.args['layerType'] # 绘制类型 (polygon、marker 等)
    coords = e.args['layer'].get('_latlng') or e.args['layer'].get('_latlngs') # 坐标
    ui.notify(f'绘制了 {layer_type}, 坐标: {coords}')

# 配置绘制工具条：启用所有绘制类型，支持编辑和删除
draw_config = {
    'draw': {
        'polygon': True,
        'marker': True,
        'circle': True,
        'rectangle': True,
        'polyline': True,
        'circlemarker': True,
    },
    'edit': {'edit': True, 'remove': True}
}

m = ui.leaflet(center=(51.505, -0.09), draw_control=draw_config).classes('h-96')
m.on('draw:created', handle_draw_create) # 绘制完成事件
m.on('draw:edited', lambda: ui.notify('编辑完成')) # 编辑完成事件
```

```
m.on('draw:deleted', lambda: ui.notify('删除完成')) # 删除完成事件
```

示例：自定义绘制样式

通过 `hide_drawn_items=True` 隐藏默认绘制结果，再通过 `generic_layer` 自定义渲染。

```
def handle_draw(e: events.GenericEventArguments):
    # 自定义多边形样式（红色、线宽 1）
    custom_options = {'color': 'red', 'weight': 1}
    # 从事件参数中获取绘制的坐标，创建自定义图层
    m.generic_layer(name='polygon', args=[e.args['layer']['_latlngs'], custom_options])

# 仅启用多边形绘制，禁用编辑和删除
draw_config = {
    'draw': {'polygon': True, 'marker': False, 'circle': False, 'rectangle': False,
    'polyline': False, 'circlemarker': False},
    'edit': {'edit': False, 'remove': False}
}

m = ui.leaflet(center=(51.5, 0), draw_control=draw_config,
hide_drawn_items=True).classes('h-96')
m.on('draw:created', handle_draw)
```

五、进阶用法

1. 等待地图初始化

地图加载需要时间，通过 `await m.initialized()` 确保在地图完全初始化后执行后续操作（如适配图层边界）。

示例：初始化后适配多边形边界

```
@ui.page('/')
async def page():
    m = ui.leaflet(zoom=5).classes('h-96')
    # 添加中央公园多边形
    central_park = m.generic_layer(name='polygon', args=[[
        (40.767809, -73.981249),
        (40.800273, -73.958291),
        (40.797011, -73.949683),
        (40.764704, -73.973741),
    ]])

    await m.initialized() # 等待地图初始化完成
    # 获取多边形边界并适配地图视图
    bounds = await central_park.run_method('getBounds')
    m.run_map_method('fitBounds', [[bounds['_southwest'], bounds['_northEast']]])

ui.run()
```

2. 集成 Leaflet 插件

通过 `additional_resources` 参数加载插件的 CSS/JS 文件，扩展地图功能。

示例：加载 RotatedMarker 插件（支持标记点旋转）

```
# 加载旋转标记点插件
m = ui.leaflet(center=(51.51, -0.09), additional_resources=[
    'https://unpkg.com/leaflet-rotatedmarker@0.2.0/leaflet.rotatedMarker.js',
]).classes('h-64')

# 添加带旋转角度的标记点（rotationAngle 单位：度）
m.marker(latlng=(51.51, -0.091), options={'rotationAngle': -30}) # 逆时针旋转 30 度
m.marker(latlng=(51.51, -0.090), options={'rotationAngle': 30})  # 顺时针旋转 30 度
```

3. 绑定组件与地图状态

通过 `bind_text_from` 等绑定方法，实现组件与地图状态（如中心点、缩放级别）的实时同步。

示例：实时显示地图状态

```
m = ui.leaflet(center=(51.505, -0.09), zoom=13).classes('h-64')

# 绑定中心点显示
ui.label('中心点: ').bind_text_from(m, 'center', lambda c: f'({c[0]:.3f}, {c[1]:.3f})')
# 绑定缩放级别显示
ui.label('缩放级别: ').bind_text_from(m, 'zoom', lambda z: str(z))

# 滑块控制缩放级别
ui.slider(min=1, max=18, value=m.zoom).bind_value_to(m, 'zoom')
```

六、版本特性与注意事项

1. 版本新增功能

版本	新增特性
2.0.0	新增 <code>hide_drawn_items</code> 参数，支持隐藏绘制结果
2.11.0	新增 <code>additional_resources</code> 参数，支持加载插件
2.12.0	支持 WMTS 和 WMS 地图服务
2.16.0	新增 <code>html_id</code> 属性，用于定位 DOM 元素
2.17.0	新增 <code>image_overlay</code> 和 <code>video_overlay</code> 方法
2.18.0	支持同时指定 Python 事件处理器和 JS 处理器

2. 注意事项

- 版权声明：使用第三方地图服务（如 OpenStreetMap、OpenTopoMap）时，需按要求保留 `attribution` 版权声明。
- 事件区分：地图点击事件为 `'map-click'`，而组件容器的点击事件为 `'click'`，避免混淆。
- 异步操作：调用 `run_map_method`、`run_layer_method` 等涉及 Leaflet 原生方法的操作时，若需获取返回值，需使用 `await` 异步等待。
- 插件兼容性：加载 Leaflet 插件时，需确保插件版本与 Leaflet 核心库兼容（NiceGUI 封装的 Leaflet 版本需参考官方文档）。

七、核心方法速查表

方法	说明	参数
<code>set_center(center)</code>	设置地图中心点	<code>center</code> ：元组（纬度，经度）
<code>set_zoom(zoom)</code>	设置缩放级别	<code>zoom</code> ：整数（1-18）
<code>marker(latlng, options)</code>	添加标记点	<code>latlng</code> ：坐标元组； <code>options</code> ：标记点配置（如图标、旋转角度）
<code>tile_layer(url_template, options)</code>	添加 WMTS 瓦片图层	<code>url_template</code> ：瓦片 URL 模板； <code>options</code> ：图层配置
<code>wms_layer(url_template, options)</code>	添加 WMS 图层	<code>url_template</code> ：WMS 服务 URL； <code>options</code> ：图层配置（如 <code>layers</code> ）
<code>image_overlay(url, bounds, options)</code>	添加图像叠加层	<code>url</code> ：图片 URL； <code>bounds</code> ：边界坐标； <code>options</code> ：透明度等
<code>video_overlay(url, bounds, options)</code>	添加视频叠加层	<code>url</code> ：视频 URL； <code>bounds</code> ：边界坐标； <code>options</code> ：自动播放等
<code>generic_layer(name, args)</code>	添加矢量图层	<code>name</code> ：图层类型（circle/polygon 等）； <code>args</code> ：图层参数
<code>initialized()</code>	等待地图初始化	-
<code>run_map_method(name, *args)</code>	调用地图原生方法	<code>name</code> ：方法名； <code>args</code> ：方法参数
<code>run_layer_method(layer_id, name, *args)</code>	调用图层原生方法	<code>layer_id</code> ：图层 ID； <code>name</code> ：方法名； <code>args</code> ：方法参数

总结

`ui.leaflet` 是 NiceGUI 中功能强大的地理信息可视化组件，通过简洁的 Python API 封装了 Leaflet 的核心能力，支持多源地图服务、丰富的图层类型和灵活的交互操作。适用于快速开发地图标注、地理数据可视化、路径规划等应用场景，无需深入学习前端 Leaflet 开发即可实现高质量的交互式地图功能。通过插件扩展和原生方法调用，还可满足更复杂的定制化需求。